DeliverMe App Logic Flows

This document maps the active app flows between the React Native frontend and the Node/Express backend. It excludes `node_modules` and the draft folder (`backend/Drafts`).

For socket-only details, see `socket.md`. This file focuses on screen logic, REST API calls, backend controllers, database/queue side effects, and example payloads.

Index

Setup And Reference

- Base URLs
- API Endpoint Summary

Shared App Flows

- Flow 1: App Launch And Session Restore
- Flow 15: Logout

Client Flows

- Flow 2: Client Signup
- Flow 3: Client Verification
- Flow 4: Client Signin
- Flow 8: Client Chooses Pickup And Destination
- Flow 9: Client Requests A Ride
- Flow 10: Searching For Nearby Drivers After Ride Request

Driver Flows

- Flow 5: Driver Signup
- Flow 6: Driver Verification
- Flow 7: Driver Signin
- Flow 11: Driver Goes Online
- Flow 12: Driver Background Location And Heartbeat
- Flow 13: Driver Receives Ride Requests
- Flow 14: Driver Goes Offline

Backend Worker And Debug Flows

- Flow 16: Ride Expiration Worker
- Flow 17: Redis Debug Routes

Type Reference And Notes

- Data Types
- Notes From Review

Base URLs

Frontend API base is defined in `frontend/src/services/api.js`:

```
const BASE_URL = "http://10.114.106.200:5000/api/auth";
const API_ROOT = "http://10.114.106.200:5000/api";
```

Backend route mounting is defined in `backend/src/app.mjs`:

```
app.use('/api/auth', authRoutes);
app.use('/api/debug/redis', redisDebugRoutes);
app.use('/api/rides', ridesRoutes);
```

Protected endpoints require:

```
Authorization: Bearer <JWT_TOKEN>
```

The JWT is stored by the frontend as:

```
AsyncStorage["userToken"] = string;
AsyncStorage["userType"] = "client" | "driver";
AsyncStorage["userData"] = JSON.stringify({ name: string, mobile: string });
```

API Endpoint Summary

Flow	Method	Endpoint	Frontend caller	Backend handler
Client signup	`POST`	`/api/auth/client/signup`	`clientSignup()` in `frontend/src/services/api.js:49`	Route `backend/src/routes/authRoutes.mjs:46`, controller `clientSignUp()` in `backend/src/controllers/authController.mjs:213`
Client verify	`POST`	`/api/auth/client/verify`	`verifyClient()` in `frontend/src/services/api.js:108`	Route `backend/src/routes/authRoutes.mjs:50`, controller `verifyClient()` in `backend/src/controllers/authController.mjs:313`

Flow	Method	Endpoint	Frontend caller	Backend handler
Client signin	`POST`	`/api/auth/client/signin`	`clientSignin()` in `frontend/src/services/api.js:76`	Route `backend/src/routes/authRoutes.mjs:55`, controller `clientSignIn()` in `backend/src/controllers/authController.mjs:359`
Driver signup	`POST`	`/api/auth/driver/signup`	`driverSignup()` in `frontend/src/services/api.js:135`	Route `backend/src/routes/authRoutes.mjs:62`, upload middleware `backend/src/middlewares/uploadMiddleware.mjs`, controller `driverSignUp()` in `backend/src/controllers/authController.mjs:409`
Driver verify	`POST`	`/api/auth/driver/verify`	`verifyDriver()` in `frontend/src/services/api.js:207`	Route `backend/src/routes/authRoutes.mjs:67`, controller `verifyDriver()` in `backend/src/controllers/authController.mjs:586`
Driver signin	`POST`	`/api/auth/driver/signin`	`driverSignin()` in `frontend/src/services/api.js:177`	Route `backend/src/routes/authRoutes.mjs:72`, controller `driverSignIn()` in `backend/src/controllers/authController.mjs:756`
Client request ride	`POST`	`/api/auth/client/request-ride`	`requestRide()` in `frontend/src/services/api.js:236`	Route `backend/src/routes/authRoutes.mjs:78`
Driver availability	`PATCH`	`/api/auth/driver/availability`	`updateDriverAvailability()` in `frontend/src/services/api.js:266`	Route `backend/src/routes/authRoutes.mjs:190`, controller `updateDriverAvailability()` in `backend/src/controllers/authController.mjs:642`
Nearby drivers for ride	`GET`	`/api/rides/:rideId/nearby-drivers?radiusKm=5&limit=20`	`getNearbyDrivers()` in `frontend/src/services/api.js:301`	Route `backend/src/routes/ridesRoutes.mjs:9`
Auth health	`GET`	`/api/auth/`	No active frontend caller found	Route `backend/src/routes/authRoutes.mjs:203`
API health	`GET`	`/`, `/health`, `/ready`	No active frontend caller found	Routes in `backend/src/app.mjs`
Redis debug	`GET`	`/api/debug/redis/...`	No active frontend caller found	`backend/src/routes/redisDebugRoutes.mjs`

Shared App Flows

Flow 1: App Launch And Session Restore

Frontend files:

- `frontend/src/screens/HomeScreen.js`
- `frontend/src/screens/ClientSigninScreen.js`

- `frontend/src/screens/DriverSigninScreen.js`
- `frontend/src/screens/DriverSignupScreen.js`
- `frontend/src/navigation/AppNavigator.js`

Steps:

1. Screen reads `userToken` and `userType` from `AsyncStorage`.
2. If both exist, it decodes the JWT using `jwtDecode`.
3. If `decoded.exp` is expired or decoding fails, it clears `AsyncStorage`.
4. If token is valid:

- `userType === "driver"` -> navigate to `DriverHome`
- otherwise -> navigate to `ClientHome`

1. `AppNavigator` listens for internal `SESSION_EXPIRED` app events and resets navigation to `Home`.

No backend request is made during session restore. It is local token validation only.

Stored data example:

```
{
  "userToken": "eyJhbGciOiJIUzI1NiIs...",
  "userType": "driver",
  "userData": "{\"name\":\"Mona Driver\",\"mobile\":\"01012345678\"}"
}
```

Client Account Flows

Flow 2: Client Signup

Frontend files:

- Screen: `frontend/src/screens/ClientSignupScreen.js`
- API function: `clientSignup()` in `frontend/src/services/api.js:49`

Backend files:

- Route: `backend/src/routes/authRoutes.mjs:46`
- Controller: `clientSignUp()` in `backend/src/controllers/authController.mjs:213`

Frontend validation:

- `email`, `name`, `password`, and `mobile` are required.
- Email must match a basic email regex.
- Password must be at least 8 characters and contain uppercase, lowercase, and a special character.
- Mobile must be exactly 11 digits.

Request:

```
POST /api/auth/client/signup
Content-Type: application/json
```

```
{
  "email": "client@example.com",
  "mobile": "01012345678",
  "name": "Client User",
  "password": "Password@123"
}
```

Backend logic:

1. Starts a MongoDB session/transaction.
2. Checks `clients` collection by `mobile`.
3. If client exists and is verified, returns `"Client already verified"`.
4. If client exists but is not verified, reuses or creates `verificationCode`, emails it, and returns a verification-required message.
5. If new client:

- Hashes password with bcrypt.
- Generates a 6-digit verification code.
- Inserts client with `clientVerified: false`.
- Sends verification email.
- Commits transaction.

1. If an error occurs, aborts transaction and deletes the inserted client document matching `{ email, mobile }`.

Success response examples:

```
{
  "message": "Client registered successfully. Please verify your email."
}
```

```
{
  "message": "Client already exists, an email with verification code sent to your email"
}
```

```
{
  "message": "Client already verified"
}
```

Frontend after response:

- For registration or unverified existing client, `ClientSignupScreen` shows the verification code UI.
- For already verified client, it navigates to `ClientSignin`.

Flow 3: Client Verification

Frontend files:

- `frontend/src/screens/ClientSignupScreen.js`
- `frontend/src/screens/ClientSigninScreen.js`
- API function: `verifyClient()` in `frontend/src/services/api.js:108`

Backend files:

- Route: `backend/src/routes/authRoutes.mjs:50`
- Controller: `verifyClient()` in `backend/src/controllers/authController.mjs:313`

Request:

```
POST /api/auth/client/verify
Content-Type: application/json
```

```
{
  "mobile": "01012345678",
  "verificationCode": "123456"
}
```

Backend logic:

1. Finds client by `mobile`.
2. If not found, returns `400`.
3. If verification code does not match:

- Sends the existing code again by email.
- Returns status `200` with a failure message.

1. If code matches:

- Updates `clientVerified` to `true`.
- Generates JWT with role `"client"`.
- Returns token and client profile.

Success response:

```
{
  "message": "Client verified successfully",
  "clientVerified": true,
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "client": {
    "name": "Client User",
    "mobile": "01012345678"
  }
}
```

Wrong-code response:

```
{
  "message": "Wrong verification code, please check your email"
}
```

Frontend after success:

```
AsyncStorage["userToken"] = response.token;
AsyncStorage["userType"] = "client";
AsyncStorage["userData"] = JSON.stringify(response.client);
```

Then navigation resets to `ClientHome`.

Flow 4: Client Signin

Frontend files:

- Screen: `frontend/src/screens/ClientSigninScreen.js`
- API function: `clientSignin()` in `frontend/src/services/api.js:76`

Backend files:

- Route: `backend/src/routes/authRoutes.mjs:55`
- Controller: `clientSignIn()` in `backend/src/controllers/authController.mjs:359`

Request:

```
POST /api/auth/client/signin
Content-Type: application/json
```

```
{
  "mobile": "01012345678",
  "password": "Password@123"
}
```

Backend logic:

1. Finds client by `mobile`.
2. Compares password with bcrypt.
3. If not verified, returns `clientVerified: false` and email.
4. If verified, generates JWT with role `"client"`.

Verified success response:

```
{
  "message": "Sign-in successful",
  "clientVerified": true,
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "client": {
    "name": "Client User",
    "mobile": "01012345678"
  }
}
```

Verification-required response:

```
{
  "message": "Verification required",
  "clientVerified": false,
  "email": "client@example.com"
}
```

Frontend after verified signin:

1. Clears `AsyncStorage`.
2. Stores `userToken`, `userType: "client"`, and `userData`.
3. Resets navigation to `ClientHome`.

Frontend after unverified signin:

- Shows verification-code UI on the signin screen.
- Calls the same `/client/verify` endpoint when the user submits the code.

Driver Account Flows

Flow 5: Driver Signup

Frontend files:

- Screen: `frontend/src/screens/DriverSignupScreen.js`
- API function: `driverSignup()` in `frontend/src/services/api.js:135`

Backend files:

- Route: `backend/src/routes/authRoutes.mjs:62`
- Upload middleware: `backend/src/middlewares/uploadMiddleware.mjs`
- Controller: `driverSignUp()` in `backend/src/controllers/authController.mjs:409`

Frontend validation:

- Required text fields:
- `email`
- `mobile`
- `name`
- `password`
- `vehicleModel`
- `vehicleColor`
- `vehiclePlateNumber`
- Required files:
- `license`
- `registration`
- `criminal`
- `personal`
- Email/password/mobile rules match client signup.
- Frontend file picker limits selected image size to 1MB.

Request:

```
POST /api/auth/driver/signup
Content-Type: multipart/form-data
```

Form data example:

```
email=driver@example.com
mobile=01099998888
name=Driver User
password=Password@123
vehicleModel=Toyota Corolla
vehicleColor=White
vehiclePlateNumber=ABC-123
license=@license.jpg
registration=@registration.jpg
criminal=@criminal.jpg
personal=@personal.jpg
```

Frontend file conversion in `driverSignup()`:

```
formData.append("license", {
  uri: data.license,
  type: "image/jpeg",
  name: "license.jpg"
});
```

Backend upload middleware:

- Uses multer memory storage.
- Accepts extensions: `jpg`, `jpeg`, `png`, `pdf`.
- Max backend file size: 5MB per file.
- Expects one file for each field: `license`, `registration`, `criminal`, `personal`.

Backend logic:

1. Starts MongoDB transaction.
2. Checks `drivers` collection by `mobile`.
3. If driver exists and verified, returns `"Driver already verified"`.
4. If driver exists but not verified, emails/reuses verification code.
5. If new driver:

- Hashes password.
- Generates verification code.
- Creates a Google Drive folder named by mobile.
- Uploads each document file to that folder.
- Inserts driver document into MongoDB with vehicle and `driverPhotos`.
- Sends verification email.
- Commits transaction.

1. On failure:

- Aborts transaction.
- Deletes Google Drive folder if it was created.
- Deletes driver document matching `{ email, mobile }`.

Success response examples:

```
{
  "message": "Driver registered successfully. Please verify your email."
}
```

```
{
  "message": "Driver already exists, an email with verification code sent to your email"
}
```

```
{
  "message": "Driver already verified"
}
```

Frontend after response:

- For registration or unverified existing driver, `DriverSignupScreen` shows verification UI.
- For already verified driver, it navigates to `DriverSignin`.

Flow 6: Driver Verification

Frontend files:

- `frontend/src/screens/DriverSignupScreen.js`
- `frontend/src/screens/DriverSigninScreen.js`
- API function: `verifyDriver()` in `frontend/src/services/api.js:207`

Backend files:

- Route: `backend/src/routes/authRoutes.mjs:67`
- Controller: `verifyDriver()` in `backend/src/controllers/authController.mjs:586`

Request:

```
POST /api/auth/driver/verify
Content-Type: application/json
```

```
{
  "mobile": "01099998888",
  "verificationCode": "123456"
}
```

Backend logic:

1. Finds driver by `mobile`.
2. If code is wrong, resends current verification code by email.
3. If code is correct, updates `driverVerified` to `true`.
4. Generates JWT with role `"driver"`.
5. Returns token and driver profile.

Success response:

```
{
  "message": "Driver verified successfully",
  "driverVerified": true,
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "driver": {
    "name": "Driver User",
    "mobile": "01099998888"
  }
}
```

Frontend after success:

```
AsyncStorage["userToken"] = response.token;
AsyncStorage["userType"] = "driver";
AsyncStorage["userData"] = JSON.stringify(response.driver);
```

Then navigation resets to `DriverHome`.

Flow 7: Driver Signin

Frontend files:

- Screen: `frontend/src/screens/DriverSigninScreen.js`
- API function: `driverSignin()` in `frontend/src/services/api.js:177`

Backend files:

- Route: `backend/src/routes/authRoutes.mjs:72`
- Controller: `driverSignIn()` in `backend/src/controllers/authController.mjs:756`

Request:

```
POST /api/auth/driver/signin
Content-Type: application/json
```

```
{
  "mobile": "01099998888",
  "password": "Password@123"
}
```

Backend logic:

1. Finds driver by `mobile`.
2. Compares password with bcrypt.
3. If not verified:

- Sends verification email again.
- Returns `driverVerified: false`.

1. If verified:

- Generates JWT with role `"driver"`.
- Returns token and driver profile.

Verified success response:

```
{
  "message": "Sign-in successful",
  "driverVerified": true,
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "driver": {
    "name": "Driver User",
    "mobile": "01099998888"
  }
}
```

Verification-required response:

```
{
  "message": "Verification required",
  "driverVerified": false,
  "email": "driver@example.com"
}
```

Frontend after verified signin:

1. Clears `AsyncStorage`.
2. Stores `userToken`, `userType: "driver"`, and `userData`.
3. Resets navigation to `DriverHome`.

Client Ride Flows

Flow 8: Client Chooses Pickup And Destination

Frontend files:

- `frontend/src/screens/ClientHomeScreen.js`
- `frontend/src/screens/MapPickerScreen.js`

This flow is local/mobile only. No backend API call is made until the user presses "Find a driver".

Steps:

1. `ClientHomeScreen` requests foreground location permission.
2. It gets current GPS position using Expo Location.
3. It reverse-geocodes the current location and sets it as the default pickup.
4. For pickup/destination edits, it navigates to `MapPicker`.
5. `MapPickerScreen` lets the user tap the map.
6. It reverse-geocodes the selected coordinate.
7. It calls the `onSelect(coords, addressText)` callback passed through navigation params.
8. `ClientHomeScreen` stores selected coordinates and addresses.
9. `MapViewDirections` calculates `routeDistance` in kilometers.

Coordinate shape used by frontend:

```
{
  "latitude": 30.0444,
  "longitude": 31.2357,
  "latitudeDelta": 0.01,
  "longitudeDelta": 0.01
}
```

Flow 9: Client Requests A Ride

Frontend files:

- Screen: `frontend/src/screens/ClientHomeScreen.js`
- API function: `requestRide()` in `frontend/src/services/api.js:236`

- Next screen: `frontend/src/screens/SearchingDriverScreen.js`

Backend files:

- Route: `backend/src/routes/authRoutes.mjs:78`
- Auth middleware: `backend/src/middlewares/auth.mjs`
- Redis helper: `addRideToGeo()` in `backend/src/redis/redisClient.mjs`
- Queue: `rideQueue` in `backend/src/queues/rideQueue.mjs`
- Worker: `backend/src/workers/rideMatching.worker.mjs`

Frontend requirements before request:

- `pickupCoords` exists.
- `destinationCoords` exists.
- `fare` exists.
- `routeDistance` is finite and greater than 0.
- Fare must be at least `routeDistance * KM_RATE`.

Request:

```
POST /api/auth/client/request-ride
Content-Type: application/json
Authorization: Bearer <CLIENT_JWT>
```

```
{
  "pickup": {
    "latitude": 30.0444,
    "longitude": 31.2357,
    "latitudeDelta": 0.01,
    "longitudeDelta": 0.01
  },
  "destination": {
    "latitude": 30.0600,
    "longitude": 31.2500,
    "latitudeDelta": 0.01,
    "longitudeDelta": 0.01
  },
  "pickupAddress": "Tahrir Square Cairo",
  "destinationAddress": "Nasr City Cairo",
  "fare": 120,
  "routeDistance": 8.5
}
```

Backend validation:

- Requires valid JWT.

- Requires `pickup`, `destination`, `fare`, and `clientId`.
- Reads `clientName` and `clientMobile` from the verified JWT (`req.user.name` and `req.user.mobile`), not from the request body.
- `fare` must be a number greater than 0.
- `pickup` and `destination` must be objects.
- Backend calculates haversine distance in meters.
- Backend accepts the request only if calculated meters are less than `routeDistance * 1000 * 1.2`.

Backend ride document:

```
{
  "clientId": "ObjectId(clientId)",
  "pickup": {
    "type": "Point",
    "coordinates": [31.2357, 30.0444]
  },
  "destination": {
    "type": "Point",
    "coordinates": [31.25, 30.06]
  },
  "pickupAddress": "Tahrir Square Cairo",
  "destinationAddress": "Nasr City Cairo",
  "clientName": "Client User",
  "clientMobile": "01012345678",
  "fare": 120,
  "status": "pending",
  "assignedDriverId": null,
  "createdAt": "Date",
  "expiresAt": "Date + 2 minutes",
  "routeDistance": 8500
}
```

Backend side effects:

1. Inserts ride into MongoDB `rides`.
2. Adds ride to Redis geo/cache through `addRideToGeo()`, including `clientName` and `clientMobile`.
3. Adds BullMQ job:

```
rideQueue.add("matchRide", { rideId });
```

1. Responds immediately to frontend.
2. Worker later finds nearby drivers and emits `ride_request` socket event to each driver room.

Success response:

```
{
  "message": "Ride requested successfully",
  "rideId": "65f1b123e4b0a12345678902"
}
```

Frontend after success:

- Navigates to `SearchingDriver`.
- Passes `rideId` in navigation params.

Flow 10: Searching For Nearby Drivers After Ride Request

Frontend files:

- Screen: `frontend/src/screens/SearchingDriverScreen.js`
- API function: `getNearbyDrivers()` in `frontend/src/services/api.js:301`

Backend files:

- Route: `backend/src/routes/ridesRoutes.mjs:9`
- Auth middleware: `backend/src/middlewares/auth.mjs`
- Redis helper: `findNearbyDrivers()` in `backend/src/redis/redisClient.mjs`

Request:

```
GET /api/rides/65f1b123e4b0a12345678902/nearby-drivers?radiusKm=5&limit=20
Authorization: Bearer <CLIENT_JWT>
```

Backend logic:

1. Validates JWT.
2. Validates `rideId` as MongoDB ObjectId.
3. Loads ride from MongoDB.
4. If requester is a client, checks that `ride.clientId` matches `req.user.id`.
5. Reads pickup GeoJSON coordinates from ride.
6. Calls `findNearbyDrivers(longitude, latitude, radiusKm, limit)`.
7. Returns nearby alive drivers from Redis, enriched with cached `driverName`, `driverMobile`, and `vehicle`.

Success response:

```
{
  "count": 2,
  "drivers": [
    {
      "driverId": "65f1a7c8e4b0a12345678901",
      "distanceKm": 1.32,
      "driverName": "Driver User",
      "driverMobile": "01099998888",
      "vehicle": {
        "model": "Toyota Corolla",
        "color": "White",
        "plateNumber": "ABC-123"
      }
    },
    {
      "driverId": "65f1a7c8e4b0a12345678903",
      "distanceKm": 3.08,
      "driverName": "Second Driver",
      "driverMobile": "01088887777",
      "vehicle": {
        "model": "Hyundai Elantra",
        "color": "Black",
        "plateNumber": "XYZ-789"
      }
    }
  ]
}
```

Frontend after response:

- Shows driver name, mobile, vehicle, and distance in `SearchingDriverScreen`.
- If none are returned, shows "No available drivers yet."

Important: this screen polls once on mount. Live driver ride notifications are handled by sockets on the driver side, documented in `socket.md`.

Driver Runtime Flows

Flow 11: Driver Goes Online

Frontend files:

- Screen: `frontend/src/screens/DriverHomeScreen.js`
- API function: `updateDriverAvailability()` in `frontend/src/services/api.js:266`

- Background tracking: `frontend/src/services/backgroundLocationService.js`
- Socket helper: `frontend/src/services/DriverSocket.js`

Backend files:

- Route: `backend/src/routes/authRoutes.mjs:190`
- Controller: `updateDriverAvailability()` in `backend/src/controllers/authController.mjs:642`
- Queue: `driverQueue` in `backend/src/queues/driverQueue.mjs`
- Worker: `backend/src/workers/rideMatching.worker.mjs`

Frontend steps:

1. User toggles availability to online in `DriverHomeScreen`.
2. App requires location permissions.
3. App gets current GPS position.
4. App initializes socket and attaches ride listeners.
5. App calls REST API `PATCH /driver/availability`.
6. App starts background location tracking.
7. App emits socket events:

- `driver:register`
- `driverOnline`
- `driver:availability`

REST request:

```
PATCH /api/auth/driver/availability
Content-Type: application/json
Authorization: Bearer <DRIVER_JWT>
```

```
{
  "available": true,
  "location": {
    "latitude": 30.0444,
    "longitude": 31.2357
  }
}
```

Backend REST logic:

1. Validates JWT.
2. Reads driver ID from `req.user.id`.
3. If `available === true`, validates `location.latitude` and `location.longitude`.
4. Updates MongoDB driver:

```
{
  "isAvailable": true,
  "updatedAt": "Date",
  "location": {
    "type": "Point",
    "coordinates": [31.2357, 30.0444]
  }
}
```

1. Loads driver data.
2. Adds BullMQ job to `driver-availability` queue:

```
driverQueue.add("driver-online", {
  driverId,
  available: true,
  driverName: "Driver User",
  driverMobile: "01099998888",
  driverVehicle: {
    "model": "Toyota Corolla",
    "color": "White",
    "plateNumber": "ABC-123"
  },
  location: {
    "latitude": 30.0444,
    "longitude": 31.2357
  },
  timestamp: 1777626000000
});
```

1. Responds immediately.

Success response:

```
{
  "message": "Availability updated successfully"
}
```

Driver availability worker side effects:

1. Adds driver data to Redis.
2. Finds nearby pending rides using driver location.
3. Publishes Redis message to `driver:rides`.
4. Socket bridge emits `nearby-rides` to `driver:{driverId}`.

Socket and background tracking details are in `socket.md`.

Flow 12: Driver Background Location And Heartbeat

Frontend files:

- `frontend/src/services/backgroundLocationService.js`
- `frontend/src/services/DriverSocket.js`

Backend files:

- `backend/src/socket/driver.socket.mjs`
- `backend/src/redis/redisClient.mjs`

This is socket-driven rather than REST-driven, but it is part of the driver online logic.

Steps:

1. `startBackgroundLocationTracking()` starts Expo location updates.
2. Background task checks `AsyncStorage["driverAvailable"]`.
3. If driver is offline, it skips update.
4. If no location data exists, it sends heartbeat.
5. If first location or moved at least 2000 meters, it sends `driverLocation`.
6. If movement is below threshold, it sends `driverHeartbeat`.

Location payload:

```
{
  "latitude": 30.0444,
  "longitude": 31.2357,
  "accuracy": 15
}
```

Backend side effects:

- `driverLocation` updates `drivers:geo`, `driver:{id}:alive`, `driver:{id}:online`, and `driver:{id}:socket`.
- `driverHeartbeat` refreshes presence TTL keys without changing geo position.

Flow 13: Driver Receives Ride Requests

Frontend files:

- `frontend/src/screens/DriverHomeScreen.js`

Backend files:

- `backend/src/workers/rideMatching.worker.mjs`
- `backend/src/socket/SocketIndex.mjs`

Incoming events:

- `ride_request` : a single ride object from the ride matching worker.
- `nearby-rides` : an array of ride objects when a driver goes online and matching pending rides already exist nearby.

Frontend logic:

1. Driver screen attaches listeners for both events.
2. `nearby-rides` replaces the current request list with a deduped array.
3. `ride_request` appends one ride to the current list, then dedupes.
4. UI displays each request with `clientName` , `_id` , `pickupAddress` , and `fare` .
5. `clientMobile` is present in the ride payload/cache for backend use, but `DriverHomeScreen` does not display it.

Ride payload example:

```
{
  "_id": "65f1b123e4b0a12345678902",
  "clientId": "65f1a7c8e4b0a12345678901",
  "clientName": "Client User",
  "clientMobile": "01012345678",
  "pickupAddress": "Tahrir Square Cairo",
  "destinationAddress": "Nasr City Cairo",
  "fare": 120,
  "status": "pending",
  "pickup": {
    "type": "Point",
    "coordinates": [31.2357, 30.0444]
  },
  "destination": {
    "type": "Point",
    "coordinates": [31.25, 30.06]
  },
  "expiresAt": "2026-05-01T09:02:00.000Z"
}
```

Current limitation:

- The "Accept Ride" button renders in `DriverHomeScreen` , but no active accept-ride API call or socket event was found in the reviewed files.

Flow 14: Driver Goes Offline

Frontend files:

- `frontend/src/screens/DriverHomeScreen.js`
- `frontend/src/services/api.js`
- `frontend/src/services/backgroundLocationService.js`
- `frontend/src/services/SocketManager.js`

Backend files:

- `backend/src/controllers/authController.mjs`
- `backend/src/workers/rideMatching.worker.mjs`
- `backend/src/socket/driver.socket.mjs`

Frontend steps:

1. User toggles availability to offline.
2. App calls:

```
PATCH /api/auth/driver/availability
Content-Type: application/json
Authorization: Bearer <DRIVER_JWT>
```

```
{
  "available": false,
  "location": null
}
```

1. App emits socket event:

```
{
  "isAvailable": false
}
```

1. App stops background location tracking.
2. App closes socket.
3. App stores `driverAvailable = "false"`.
4. App clears current request list.

Backend REST logic:

1. Updates MongoDB driver `isAvailable` to `false`.
2. Adds BullMQ job:

```
driverQueue.add("driver-offline", { driverId, available: false, ... });
```

1. Responds with:

```
{
  "message": "Availability updated successfully"
}
```

Worker side effect:

- Removes cached driver data and `drivers:geo` entry.
- Publishes Redis `driver:offline`.

Socket handler side effect:

- `driver:availability` with `{ isAvailable: false }` removes driver from geo and deletes presence keys:
- `driver:{id}:online`
- `driver:{id}:socket`
- `driver:{id}:alive`

Shared Logout Flow

Flow 15: Logout

Client logout file:

- `frontend/src/screens/ClientHomeScreen.js`

Driver logout file:

- `frontend/src/screens/DriverHomeScreen.js`

Searching screen logout file:

- `frontend/src/screens/SearchingDriverScreen.js`

Client logout:

1. Closes socket.
2. Clears `AsyncStorage`.
3. Shows success toast.
4. Navigates to `Home`.

Driver logout:

1. If driver is available:

- Emits socket `driver:availability` false.
- Calls `PATCH /api/auth/driver/availability` false.
- Stops background tracking.

1. Closes socket.
2. Clears `AsyncStorage`.
3. Navigates to `Home`.

Searching screen logout:

1. Clears `AsyncStorage`.

2. Closes socket.
3. Navigates to `Home`.

No dedicated backend logout endpoint exists in the reviewed files.

Backend Worker And Debug Flows

Flow 16: Ride Expiration Worker

Backend file:

- `backend/src/workers/rideExpiration.worker.mjs`

This worker is backend-only. It does not currently interact with frontend APIs.

Steps:

1. Runs every 10 seconds.
2. Finds up to 50 rides with:

```
{
  "status": "pending",
  "expiresAt": { "$lte": "now" }
}
```

1. Updates each ride to:

```
{
  "status": "expired",
  "expiredAt": "Date"
}
```

1. Removes ride from Redis geo.

Current limitation:

- Socket emits for `rideExpired` are present but commented out, so the frontend is not notified directly when a ride expires.

Flow 17: Redis Debug Routes

Backend file:

- `backend/src/routes/redisDebugRoutes.mjs`

Mounted under:

```
/api/debug/redis
```

No active frontend caller was found. These are development/debug endpoints.

Endpoints:

Method	Endpoint	Purpose
`GET`	`/api/debug/redis/drivers`	Return all driver IDs in `drivers:geo`.
`GET`	`/api/debug/redis/drivers/data`	Return cached driver hash data.
`GET`	`/api/debug/redis/driver/:driverId`	Return one driver's Redis geo coordinates.
`GET`	`/api/debug/redis/nearby?lat=30.0444&lng=31.2357&radius=5`	Find drivers near a coordinate.
`GET`	`/api/debug/redis/drivers/full`	Return drivers with coordinates.
`GET`	`/api/debug/redis/driver/:driverId/alive`	Return alive TTL state for a driver.
`GET`	`/api/debug/redis/rides/geo`	Return rides in `rides:geo` with coordinates.
`GET`	`/api/debug/redis/rides/data`	Return cached ride data strings.
`GET`	`/api/debug/redis/rides/full`	Combine ride geo position, cached data, and stale status.

Example:

```
GET /api/debug/redis/nearby?lat=30.0444&lng=31.2357&radius=5
```

```
{
  "count": 1,
  "drivers": ["65f1a7c8e4b0a12345678901"]
}
```

Data Types

Auth Payloads

```
type ClientSignupRequest = {
  email: string;
  mobile: string;
  name: string;
  password: string;
};

type ClientSigninRequest = {
  mobile: string;
  password: string;
};

type VerifyRequest = {
  mobile: string;
  verificationCode: string;
};

type AuthSuccessResponse = {
  message: string;
  token: string;
  client?: {
    name: string;
    mobile: string;
  };
  driver?: {
    name: string;
    mobile: string;
  };
};
```

Driver Signup

```
type DriverSignupRequest = FormData & {
  email: string;
  mobile: string;
  name: string;
  password: string;
  vehicleModel: string;
  vehicleColor: string;
  vehiclePlateNumber: string;
  license: File;
  registration: File;
  criminal: File;
  personal: File;
};
```

Coordinates

```
type LatLng = {
  latitude: number;
  longitude: number;
};

type FrontendMapCoords = LatLng & {
  latitudeDelta?: number;
  longitudeDelta?: number;
};

type MongoGeoPoint = {
  type: "Point";
  coordinates: [number, number]; // [longitude, latitude]
};
```

Ride Request

```
type RequestRideBody = {
  pickup: FrontendMapCoords;
  destination: FrontendMapCoords;
  pickupAddress: string;
  destinationAddress: string;
  fare: number;
  routeDistance: number; // kilometers from frontend route calculation
};

// Added by backend from the verified client JWT, not accepted from the request body.
type RideClientIdentity = {
  clientName?: string;
  clientMobile?: string;
};

type RequestRideResponse = {
  message: "Ride requested successfully";
  rideId: string;
};
```

Driver Availability

```
type UpdateDriverAvailabilityBody = {
  available: boolean;
  location: LatLng | null;
};

type UpdateDriverAvailabilityResponse = {
  message: "Availability updated successfully";
};
```

Nearby Drivers

```
type NearbyDriversResponse = {
  count: number;
  drivers: Array<{
    driverId: string;
    distanceKm: number;
    driverName: string;
    driverMobile: string;
    vehicle: {
      model?: string;
      color?: string;
      plateNumber?: string;
    } | null;
  }>;
};
```

Notes From Review

- The frontend hardcodes the API host in `frontend/src/services/api.js`.
- `driverSignin()` and `updateDriverAvailability()` pass `mode: 'no-cors'`; this is usually not useful in React Native and may hide response details in web contexts.
- Driver document file limit differs by layer: frontend checks 1MB, backend allows 5MB.
- The driver signup backend accepts `.jpg`, `.jpeg`, `.png`, and `.pdf`, while the frontend appends selected files as `type: "image/jpeg"` and filename `field.jpg`.
- Ride request matching is asynchronous: successful `POST /client/request-ride` means the ride was created and queued, not that a driver accepted it.
- No active accept-ride, cancel-ride backend API, completed-rides API, or settings API was found in the reviewed live files.
- Redis debug routes are mounted in the app and are not protected by auth in the reviewed code; they should stay development-only.